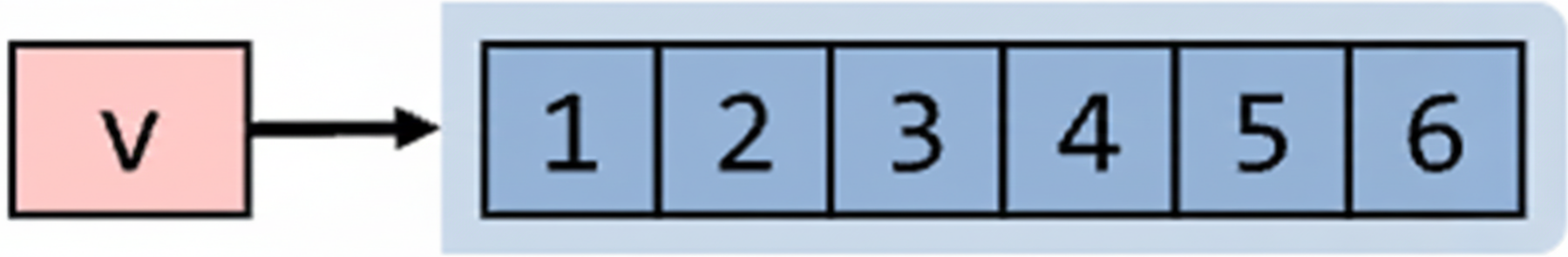


C++ QUICK REFERENCE

- Containers: store collections (vector, list, map, set, deque, ...)
- Iterators: traverse containers (input, forward, bidirectional, random-access)
- Algorithms: operate on iterator ranges (sort, find, for_each, ...)
- Functors: objects like functions, adapters (stack, queue)

std::vector

Dynamic contiguous array.



```
int main()  std::vector<int> v; for (int i = 0; i < 5; ++i)
v.push_back(i * 10);
for (std::vector<int>::size_type i = 0; i < v.size(); ++i) std ::
cout << v[i] << " "; std :: cout <<
'';
for (std::vector<int>::iterator it = v.begin(); it !=
v.end(); ++it) std::cout << *it << " "; std::cout << ";
```

std::list

Doubly-linked list.

```
int main()  std::list<int> L;
L.push_back(1); L.push_back(2); L.push_back(3); for (std :: list < int >::
const_iterator it = L.begin(); it!= L.end(); ++ it) std :: cout << *it <<
"";
```

Associative containers

```
int main()  std::map<std::string, int> ages; ages["Alice"] =
30; ages["Bob"] = 25;
for (std::map<std::string,int>::const_iterator it =
ages.begin(); it!= ages.end(); ++ it) std :: cout << it-> first << " :
```

```
" << it-> second <<
'';
```

Container adapters

```
int main()  std::queue<int> q; q.push(1); q.push(2);
q.push(3);
while (!q.empty())  std::cout << q.front() << " "; q.pop();
std::cout << ";
```

Unordered maps and pairs

```
int main()  std::map<std::string,int> ages;
ages.insert(std::make_pair(std ::
string(" Alice"), 30)); ages.insert(std :: make_pair(std ::
string(" Bob"), 25));
for (std::map<std::string,int>::const_iterator it =
ages.begin(); it!= ages.end(); ++ it) std :: cout << it-> first << " :
" << it-> second <<
'';
// If your implementation provides
unordered_map : /* include < unordered_map > std ::
unordered_map < std :: string, int > um; um[" Alice"] = 30; */
```

Iterators

```
int main()  std::vector<int> v;
v.push_back(3); v.push_back(1); v.push_back(2); std ::
sort(v.begin(), v.end()); for (std :: vector < int >:: const_iterator it =
v.begin(); it!= v.end(); ++ it) std :: cout << *it <<
' ';
```

Algorithms

Remove-Erase Idiom

```
int main()  std::vector<int> v;
v.push_back(1); v.push_back(2); v.push_back(3); v.push_back(2); v.erase(std
:: remove(v.begin(), v.end(), 2), v.end());
```

Transform and Accumulate

```
struct Square  int operator()(int x) const  return x*x;  ;
int main()  std::vector<int> v;
v.push_back(1); v.push_back(2); v.push_back(3); std :: vector < int >
squares(v.size()); std ::
transform(v.begin(), v.end(), squares.begin(), Square()); int sum =
std :: accumulate(squares.begin(), squares.end(), 0); std :: cout <<
"sum = " << sum <<
'';
```

Functors and Comparators

```
struct ByLength  bool operator()(const std::string a, const
std::string b) const  return a.size() < b.size();  ;
int main()  std::vector<std::string> v;
v.push_back("apple"); v.push_back("kiwi"); v.push_back("banana"); std ::
sort(v.begin(), v.end(), ByLength());
```

Useful Idioms

- Use `reserve()` on vector when size is known.
- Pass large containers by `const&`.
- Use `lower_bound` / `upper_bound`.

Small Example

```
struct SquareFunctor  int operator()(int x) const  return x
* x;  ;
int main()  std::vector<int> v; for (int i = 1; i <= 5; ++i)
v.push_back(i);
std::vector<int> sq(v.size()); std::transform(v.begin(),
v.end(), sq.begin(), SquareFunctor());
int sum = std::accumulate(sq.begin(), sq.end(), 0);
std::cout << "sum of squares = " << sum << " ;
std::vector<std::string> words;
words.push_back("apple"); words.push_back("kiwi"); words.push_back("banana"); std ::
map < int, int > freq; for (std :: vector < std :: string >::
const_iterator it = words.begin(); it!=
words.end(); ++ it) ++ freq[it-> size()];
for (std::map<int,int>::const_iterator it = freq.begin(); it!=
freq.end(); ++ it) std :: cout << "length" << it-> first << " :
" << it-> second << "word(s)";
```